exec.txt
TXT 608.16KB

Analise o arquivo em anexo passo a passo e determine ao final: este arquivo é benigno ou é um malware?

Análise Passo a Passo do Arquivo `exec.txt`

1. Identificação do Tipo de Arquivo

O conteúdo começa com `ELF` (DEL + "ELF"), que é a assinatura de um arquivo **ELF (Executable and Linkable Format)** — o formato padrão de executáveis e bibliotecas em Linux. O cabeçalho indica:

- **Arquitetura:** x86-64 (`>` no e_machine)
- **Tipo:** não é um executável comum — contém estruturas como `__this_module`, `init_module`, `cleanup_module`, `modinfo`, `vermagic`, `depends`, `.gnu.linkonce.this_module`, `.modinfo`, `.BTF`.

Isso caracteriza um **módulo de kernel Linux (`.ko` — kernel object)**, compilado contra `7.1.13+parrot7-amd64` (distro Parrot OS).

2. Metadados do Módulo

No bloco `.modinfo` encontramos:

```
text

author=Ciaran McNally - maK
license=GPL
depends=
name=maK_it
retpoline=Y
vermagic=7.1.13+parrot7-amd64 SMP preempt mod_unload
```

- **Nome do módulo:** `maK_it`
- **Autor:** Ciaran McNally

- **Fonte:** `maK_it.c` (caminho de compilação: `/home/user/Documents/malwares_rep/RatKeylogger/`)

⚠️ **Ponto crítico:** o diretório de origem é literalmente `malwares_rep/RatKeylogger` — ou seja, o próprio autor classificou o projeto como **"RatKeylogger"** (Remote Access Trojan + Keylogger).

3. Strings Reveladoras

Dentro do módulo encontramos strings que confirmam a funcionalidade:

3.1 Mapa de teclas (keylogger)

```
text

[ESC] ! @ # $ % ^ & * ( ) _ + [BS] [TAB] Q W E R T Y U I O P { } [ENTR] [CTRL]
A S D F G H J K L : " ~ [SHFT] | Z X C V B N M < > ? [CAPS] [F1] ... [F12]
[PSCR] [ALT] [PAUS] 1 2 3 4 6 7 8 9 0 = q w e r t y u i o p ...
```

Trata-se de uma **tabela de tradução de scancodes para caracteres** — exatamente o que um keylogger de kernel usa para converter eventos do teclado em texto legível.

3.2 Strings de operação

- `maKit: Major device failed with -1` — mensagem de erro do registro de dispositivo de caractere (`major`).
- `maK_it.c` — nome do arquivo fonte.
- `RatKeylogger` — no caminho de compilação.

3.3 Funções e símbolos importados

Observando a tabela de símbolos e relocações:

Símbolo	Significado
`__register_chrdev` , `__unregister_chrdev`	Registra/desregistra um **dispositivo de caractere** (canal de comunicação com userspace)

Símbolo	Significado
`register_keyboard_notifier`, `unregister_keyboard_notifier`	Registra um **notificador de teclado** — mecanismo clássico de keylogger em kernel
`copy_to_user`, `__check_object_size`	Copia dados do kernel para o espaço do usuário (entrega das teclas capturadas)
`keyBuffer`, `shiftPressed`, `keysShift`, `keys`	Variáveis globais do keylogger
`key_notify`, `open_dev`, `read_dev`, `init_mod`, `cleanup_module`	Funções do módulo: callback do notificador, abertura/leitura do device, init e exit
`_printk`	Log no kernel
`memset`, `strlen`, `down`, `up`	Utilitários

O par `init_mod` / `cleanup_module` e a presença de `__pfx_init_module` / `__pfx_cleanup_module` confirmam que o módulo tem inicialização e limpeza (não é um módulo passivo).

4. Estrutura de Funcionamento (reconstruída)

Com base nos símbolos, strings e tipos BTF, o módulo faz:

1. `init_mod` (`init_module`):
   - Registra um **dispositivo de caractere** via `__register_chrdev` (cria `/dev/<nome>` com um `major`).
   - Registra um **notificador de teclado** via `register_keyboard_notifier` , apontando para `key_notify` .
2. `key_notify` (callback do notificador):
   - É chamado a cada evento de teclado no kernel.
   - Usa as tabelas `keys` / `keysShift` e a variável `shiftPressed` para traduzir o scancode em caractere.
   - Armazena no buffer global `keyBuffer` .
3. `read_dev` :

- Quando o userspace lê o dispositivo (`/dev/...`), o módulo usa `copy_to_user` para entregar o conteúdo do `keyBuffer` — ou seja, **exfiltra as teclas capturadas para um processo em espaço de usuário**.

4. `cleanup_module` :
   - `unregister_keyboard_notifier + __unregister_chrdev` — desfaz o registro.

Isso é o padrão clássico de um **keylogger em modo kernel**: captura global de teclas (independente do processo/foco), armazenamento em buffer e exposição via dispositivo de caractere para um coletor em userspace (o "RAT" do nome `RatKeylogger`).

5. Evidências de Compilação Legítima vs. Intenção Maliciosa

Pontos que mostram que é um módulo real e compilado:

- Cabeçalhos de compilação do kernel Parrot (`/usr/src/linux-headers-7.1.13+parrot7-common/...`).
- Flags reais do GCC do kernel (`-mcmodel=kernel` , `-mstack-protector-guard=...` , `-mindirect-branch=thunk-extern` , `-fpatchable-function-entry=16,16` , etc.).
- Seções `.BTF` , `.orc_unwind` , `__mcount_loc` , `__jump_table` — típicas de módulos modernos.
- `license=GPL` (necessário para usar símbolos GPL-only como `register_keyboard_notifier`).

Pontos que revelam intenção maliciosa:

- Diretório de origem: `malwares_rep/RatKeylogger` .
- Nome do módulo `maK_it` (troCadilho com "make it").
- Funcionalidade inequívoca de keylogger global em kernel.
- Canal de exfiltração via chrdev para userspace.
- Autor assina como "Ciaran McNally - maK".

6. Classificação

Um módulo de kernel que:

- intercepta **todas** as teclas do sistema via `register_keyboard_notifier` ,
- converte scancodes em texto,

- acumula em buffer,
- expõe via dispositivo de caractere para leitura por userspace,

é, por definição, um **keylogger de kernel**. Combinado com o caminho `RatKeylogger` e o nome `maK_it`, não há ambiguidade sobre a intenção: é uma **ferramenta de spyware/RAT**.

Ainda que o módulo seja "bem-comportado" tecnicamente (usa APIs documentadas, licença GPL, sem rootkit de ocultação explícito nas strings analisadas), sua **função primária é a captura clandestina de entrada do usuário** — comportamento malicioso por natureza, independentemente de o autor original ter fins educacionais.

Conclusão

O arquivo é MALICIOSO.

Trata-se de um **módulo de kernel Linux (rootkit/keylogger)** chamado `maK_it`, compilado a partir de `maK_it.c` no diretório `RatKeylogger`. Ele registra um notificador de teclado no kernel para capturar globalmente todas as teclas digitadas, armazena em `keyBuffer` e expõe os dados via dispositivo de caractere (`__register_chrdev` + `copy_to_user`) para um componente em espaço de usuário — configurando um **keylogger em modo kernel**, tipicamente parte de um RAT.

Recomendação: não carregar (`insmod` / `modprobe`) este módulo em nenhum sistema. Se encontrado em uma máquina, trate como comprometimento: o módulo precisa de privilégios de root para ser inserido, então a presença dele indica que o atacante já obteve root ou enganou o administrador.

Qual é a categoria principal deste malware?

Categoria Principal do Malware

Resposta Direta

A categoria principal é **Keylogger em modo kernel (Kernel-Mode Keylogger)**, também classificado como **Spyware / RAT (Remote Access Trojan) — componente de captura de credenciais**.

Justificativa da Classificação

Evidência	Categoria que sustenta
`register_keyboard_notifier` + tabelas `keys` / `keysShift` / `shiftPressed`	**Keylogger** (captura de teclas)
Executa dentro do kernel (`__this_module` , `init_module` , `.ko`)	**Kernel-mode** (rootkit-level, invisível a antivírus de userspace)
`__register_chrdev` + `copy_to_user` + `keyBuffer`	**Canal de exfiltração** para userspace (o "RAT" coletor)
Caminho `RatKeylogger` / nome `maK_it`	Confirma a intenção: **RAT + Keylogger**
`license=GPL` , uso de símbolos GPL-only	Não é erro de licença — é deliberado para poder usar `register_keyboard_notifier`

Subcategorias Secundárias

1. **Spyware** — coleta silenciosa de entrada do usuário (credenciais, mensagens, dados sensíveis).
2. **Componente de RAT** — o módulo sozinho não se comunica com a rede; ele **expõe as teclas via** `/dev/<chrdev>` , e um processo em userspace (o RAT) lê esse dispositivo e envia pela rede. Ou seja, é a **parte de captura** de uma arquitetura RAT.
3. **Rootkit (parcial)** — opera em ring 0, o que lhe dá capacidade de ocultação e persistência. Nas strings analisadas **não há mecanismos explícitos de ocultação** (como hook de `sys_call_table` ou `hide` de módulo), então é um rootkit **funcional** (roda em kernel) mas não um rootkit **de stealth** completo — a "rootkit-ness" vem do nível de privilégio, não de técnicas de invisibilidade.

O que NÃO é

- **Não é ransomware** (nenhuma rotina de criptografia, resgate ou manipulação de arquivos).
- **Não é worm/vírus** (não há auto-propagação no código).
- **Não é botnet pura** (não há C2 embutido no módulo; a comunicação é delegada ao userspace).
- **Não é minero** (nenhuma referência a criptomoedas/hash).
- **Não é backdoor de shell** (não abre socket nem executa comandos).

Classificação Final (taxonomia)

```text
Categoria principal : Keylogger (Kernel-Mode)
Família/tipo        : Spyware / RAT component
Plataforma          : Linux (x86-64), kernel 7.1.13+parrot7-amd64
Nível de privilégio : Ring 0 (root necessário para insmod)
Vetor de entrega    : Requer root (insmod/modprobe) — não é auto-executável
Impacto             : Captura global de teclado → roubo de credenciais/dados
```

Resumo em uma frase: é um **keylogger de kernel**, componente de captura de um **RAT/spyware**, projetado para rodar em ring 0 no Linux e entregar as teclas capturadas a um coletor em espaço de usuário.